13 Testing fundamentals

This chapter covers

- Why we test
- How we test
- Test-driven development
- Test doubles
- From JUnit 4 to 5

Recent years in programming have seen a growing acceptance of automated testing as an expected part of the development process. Tests are run both locally by developers and in build and continuous integration environments to ensure our systems are behaving. Along with that has come an explosion of different tools, approaches, and philosophies.

As with any technology, there are no silver bullets—no approach to testing will cover every possible situation. Given this, it's important to understand *why* you are testing so that you can determine best *how* to test.

13.1 Why we test

In fact, that word *test* hides a multitude of possible reasons we're examining our code's behavior. A non-exhaustive (and occasionally overlapping) list to consider follows:

- Confirming the logic of an individual method is correct
- Confirming the interaction between two objects within your code
- Confirming a library or other external dependency behaves as expected
- Confirming data produced or consumed by a part of the system is valid
- Confirming a system works correctly with an external component (such as a database)
- Confirming a system's end-to-end behavior fulfills important business scenarios
- Documenting assumptions for later maintainers (because tests don't get out of sync the same way comments and documentation can)
- Influencing your system design by exposing tight coupling and object responsibilities
- Automating post release checklists that a human would otherwise execute

- Finding unexpected corner cases in code via randomized inputs

Even this short list of testing motivations shows that the simple idea of "testing your code" isn't necessarily so simple. So, when we approach testing, we need to ask ourselves the following questions:

- What is my motivation in testing this piece of code?
- Which techniques let me fulfill that goal most accurately and cleanly?

13.2 How we test

One of the most common tools in discussing different types of testing is the Testing Pyramid, shown in figure 13.1. Originally from Mike Cohn's book *Succeeding with Agile* (Addison-Wesley Professional, 2009), the pyramid expresses one way of balancing the costs of different types of testing to maximize the aid they give us.

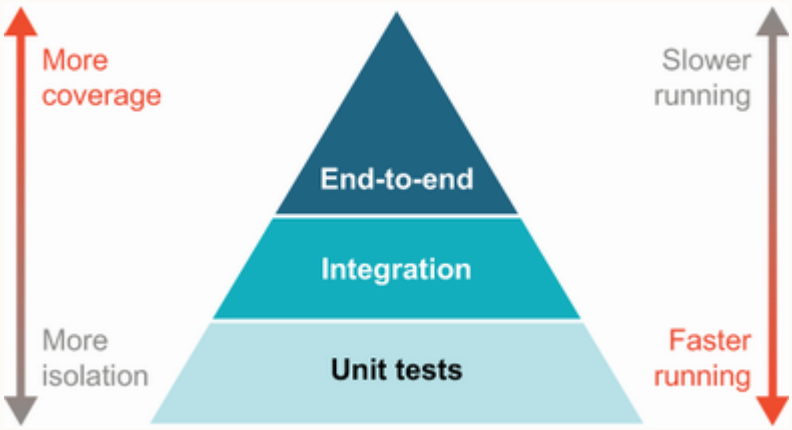


Figure 13.1 The Testing Pyramid

Although arguments rage on the internet about the exact boundaries between these types of testing, the central ideas are quite useful.

NOTE These types of tests are *not* defined by the tool you use—you aren't writing a unit test just because you are using JUnit, and using a specification library doesn't guarantee you're actually creating usable acceptance tests your stakeholders will benefit from. These types of tests are about what we want to exercise and prove.

Unit tests form the base of the pyramid. These are focused tests that exercise one aspect of a system. What do we mean, though, by "one aspect?" The easiest part is how the code under test relates to external dependencies. If your test calls the database before doing some logic on the result, that's no longer "one" thing you're testing—you're now

testing the database retrieval *and* that your logic works. Such external dependencies may also commonly include network services or files.

A common approach to avoid violating that single focus is to use *test doubles*—for example, having our unit test talk to a fake object instead of the real database. We'll discuss this in detail in the next section, but the basic idea is that this fakery has many flavors and we need to consider many necessary things if it is to be done well.

Unit tests are attractive for a number of reasons, hence their traditional placement as the largest segment in the testing pyramid. These reasons include the following:

- *Fast*—If a test has no external dependencies, it shouldn't take long to execute.
- *Focused*—By talking only about one "unit" of code, it's often clearer what the test is expressing than in larger, more setup-laden tests.
- *Reliable failure*—Minimizing external dependencies, especially dependencies on external state, helps make unit tests more deterministic.

This all sounds great, so why don't we write only unit tests all the time? The truth is that unit tests have limitations that prevent them from being useful at every scale that we need to test at. The issues include the following:

- *Tight coupling*—Because unit tests are by definition closely related to their implementation, they are also prone to binding too tightly to those implementation choices. It's not uncommon for a whole suite of unit tests to be invalidated when the underlying implementation changes.
- *Missing meaningful interactions*—Although it's attractive to think about our code as a fleet of objects all just minding their own business, the reality is that the real work of a program includes interaction between those dependent pieces, which unit tests will miss.
- *Inwardly focused*—Often the goal of testing is showing that the end users of our software get the right results. Rarely does a single method's correctness actually translate to happy users.

Integration tests, the next step up the pyramid, break free from the constraint of unit tests about talking to dependencies. Integration tests cross those boundaries and may in fact focus on ensuring different pieces of the system integrate seamlessly.

As with unit tests, integration tests may also pick and choose only parts of the system to exercise. Some dependencies, such as external services, may still be replaced by test

doubles, whereas others, such as databases, are in bounds for testing. What's key is that the tests reach out beyond a single "unit" of code in their scope.

The exact line between unit and integration tests can be blurry. Here are some examples, though, which clearly step over the line into integration testing territory:

- You need a database instance and make calls to your data access code.
- You spin up a special in-process HTTP server and test requests against it.
- You make actual calls against another service (whether or not a testing environment).

Integration tests come with a lot of nice properties, such as the following:

- *Broader coverage*—An integration test necessarily works more of your code and the code of the libraries you depend on.
- *More validation*—Certain types of errors may be detectable only when using real dependencies. For instance, a syntax error in a SQL statement will be difficult to find without calling an actual database.

Of course, no choice is without its trade-offs. Integration tests can be a source of significant pain if not managed correctly for reasons such as the following:

- *Slow tests*—Going to a real database instead of reading a value from memory is drastically slower. Multiply that by hundreds or thousands of tests, and you may find yourself waiting... a lot.
- *Nondeterministic results*—External dependencies increase the chances that important state may change between test runs. For instance, records left over in a database might alter what comes back from a SQL statement.
- *False confidence*—Integration tests sometimes use dependencies that differ subtly from those of the main system. For instance, if the test database is a different version than production, the integration test may incorrectly suggest everything is good when it isn't.

For all those difficulties, integration testing is a critical part of the testing landscape.

End-to-end tests push beyond integration tests with the aim of duplicating the full user experience of a system. This may mean programmatically driving a web browser or other application, or exercising a fully deployed instance of a service in a testing environment. End-to-end testing brings the following advantages that are difficult to duplicate at lower levels of our systems:

- *The "real" user experience*—A good end-to-end test is close to what users see. This lets us validate the users' high-level expectations directly.
- *A "real" environment*—Many end-to-end tests run against test, staging, or even production environments. This validates that our code works outside of our comfortable, carefully managed build environment.
- *UI available*—Many end-to-end testing approaches, such as those that drive web browsers, can see aspects of the system (e.g., was the button rendered, so it can be clicked) that may be difficult to validate anywhere else.

But this greater reality in our end-to-end tests comes with the next stark list of difficulties:

- *Even slower testing*—Where many unit tests run almost instantly and even integration tests can often be sub-second, an end-to-end test controlling a web browser to walk through a site will necessarily take much longer to run.
- *Flaky tests*—Historically, the tools for end-to-end testing, particularly of UIs, have been prone to flakiness, requiring retries and long timeouts to avoid unnecessary failures.
- *Brittle tests*—Because end-to-end tests live at the top of the pyramid, changes at any level below can cause failure. Seemingly innocuous text changes can unintentionally break these sprawling tests.
- *Harder debugging*—Because end-to-end testing often introduces another layer that drives the tests, figuring out what went wrong is often a chore.

With this pyramid in hand, you may be tempted to ask, "What's the right ratio of tests between the layers?" The truth is that there is no single answer. The needs of every project and system are different. But the pyramid can help guide you in the pluses and minuses of how you choose to exercise each bit of functionality in your system.

Although certainly not the only way, the well-grounded developer may find that test-driven development helps in keeping these different levels of testing clear as systems evolve.

13.3 Test-driven development

Test-driven development (TDD) has been part of the software development industry for quite some time. Its basic premise is that you write tests during your implementation rather than afterward, and those tests influence the design of your code. A commonly recommended approach to TDD, known as test first, is to actually write a failing test before providing an implementation, then refactoring as needed. For example, to write an

implementation of concatenating two string objects (`"foo"` and `"bar"`), you'd write the test first (testing the result must equal `"foobar"`) to ensure that you know your implementation is correct. Though many developers write tests, more often than not, they're writing them after the implementation and losing some major benefits of TDD.

Despite its seeming pervasiveness, many developers don't understand why they should be doing TDD. The question for many developers remains, "Why write test-driven code? What's the benefit?"

We believe that *eliminating fear and uncertainty* is the overriding reason you should write test-driven code. Kent Beck (co-inventor of the JUnit testing framework) also sums this up nicely in his book, *Test-Driven Development: By Example* (Addison-Wesley Professional, 2002):

- Fear makes you tentative.
- Fear makes you want to communicate less.
- Fear makes you shy away from feedback.
- Fear makes you grumpy.

TDD takes away the fear, making the well-grounded Java developer a more confident, communicative, receptive, and happier developer. In other words, TDD helps you break free from the mind-set that leads to statements like these:

- When starting a new piece of work, "I don't know where to start, so I'll just start hacking."
- When changing existing code, "I don't know how the existing code is going to behave, so I'm secretly too scared to change it."

TDD brings many other benefits that aren't always immediately obvious, such as the following:

- *Cleaner code*—You write only the code you need.
- *Better design*—Some developers call TDD *test-driven design*.
- *Better APIs*—Your tests serve as an additional client to the implementation, revealing rough spots early.
- *Greater flexibility*—TDD encourages coding to interfaces.
- *Documentation through tests*—Because you don't write code without a test, everything has example usage in the tests.
- *Fast feedback*—You learn about bugs *now*, not in production.

One barrier for developers who are just getting started is that TDD can sometimes be viewed as a technique that isn't used by "ordinary" developers. The perception can be that only practitioners of some imaginary "Church of Agile" or other esoteric movement use TDD, and that every TDD principle must be followed strictly to gain the benefits. This perception is completely false, as we'll demonstrate. TDD is a technique for every developer.

3.3.1 TDD in a nutshell

TDD is easiest at the unit-testing level, and if you're unfamiliar with TDD, this is a good place to start. We'll start there, but proceed to show how TDD works, in particular on the boundary of unit and integration testing.

NOTE Dealing with existing code that has very few or no tests can be a daunting task. It's almost impossible to retroactively fill in all of the tests. Instead, you should simply add tests for each new bit of functionality that you add. See Michael Feathers's excellent book *Working Effectively with Legacy Code* (Prentice Hall, 2004) for further help.

We'll start with brief coverage of the red-green-refactor premise behind TDD, using JUnit to test-drive code for calculating sales revenue for selling theater tickets. If the JUnit framework is unfamiliar, we recommend the online user guides (see **https://junit.org/junit5/docs/current/user-guide**) or, for more detail, *JUnit in Action* by Cătălin Tudose (Manning, 2020; **http://mng.bz/gwOR**). Let's start with a working example of the three basic steps of TDD—the red-green-refactor loop—by calculating the revenue when selling theater tickets.

3.3.2 A TDD example with a single use case

If you're an experienced TDD practitioner, you may want to skip this small example, although we'll offer insights that may be new. Suppose you've been asked to write a rock-solid method to calculate the revenue generated by selling a number of theater tickets. The initial business rules from the theater company's accountant are simple, as shown here:

- The baseline price of a ticket is $30.
- Total revenue = number of tickets sold * price.
- The theater seats = 100 people.

Because the theater doesn't have very good point-of-sale software, the user currently has to manually enter the number of tickets sold.

If you have practiced TDD, you'll be familiar with the three basic steps of TDD: red, green, refactor. If you're new to TDD or are looking for a little refresher, let's take a look at Kent Beck's definition of those steps, from *Test-Driven Development: By Example* :

- *Red*—Write a little test that doesn't work (failing test).
- *Green*—Make that test pass as quickly as possible (*passing test*).
- *Refactor*—Eliminate the duplication (*refined passing test*).

To give you an idea of the `TicketRevenue` implementation that we're trying to achieve, here is some pseudocode you might have in your head:

```
estimateRevenue(int numberOfTicketsSold)
  if (numberOfTicketsSold is less than 0 OR greater than 100)
    Deal with error and exit
  else
    revenue = 30 * numberOfTicketsSold;
    return revenue;
  endif
```

Note that it's important that you don't think too deeply about this. The tests will end up driving your design and partly your implementation, too.

Writing a failing test (red)

The point in this step is to start with a test that fails. In fact, the test won't even compile, because you haven't even written a `TicketRevenue` class yet!

After a brief whiteboard session with the accountant, you realize that you'll want to write tests for five cases: ticket sales that are negative, `0`, `1`, `2-100`, and `> 100`.

NOTE A good rule of thumb when writing tests (especially involving numbers) is to think of the zero/`null` case, the one case, and the many (*N*) case. A step beyond that is to think about other constraints on *N*, such as a negative amount or an amount beyond a maximum limit.

To begin, you decide to write a test that covers the revenue received from one ticket sale. Your JUnit test would look similar to the following code (remember we're not writing a perfect, passing test at this stage):

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
```

```
import java.math.BigDecimal;

import static org.junit.jupiter.api.Assertions.*;

public class TicketRevenueTest {
  private TicketRevenue venueRevenue;

  @BeforeEach
  public void setUp() {
    venueRevenue = new TicketRevenue();
  }

  @Test
  public void oneTicketSoldIsThirtyInRevenue() {    (1)
    var expectedRevenue = new BigDecimal("30");
    assertEquals(expectedRevenue, venueRevenue.estimateTotalRevenue(1));
  }
}
```

(1) One sold case

As you can see from the code, the test quite clearly expects the revenue from one ticket sale to equal 30 in revenue.

But as it stands, this test won't compile, because you haven't written a `TicketRevenue` class with the `estimateTotalRevenue(int numberOfTicketsSold)` method. To make the compilation error go away so that you can run the test, you can add a random implementation so that the test will compile, as follows:

```
public class TicketRevenue {
  public BigDecimal estimateTotalRevenue(int i) {
    return BigDecimal.ZERO;
  }
}
```

You may also find it a bit odd that the test extracts a mutable `venueRevenue` field when our general advice has been to favor immutability. The reasoning behind this is that the shared field allows us to express a common setup between our (soon-to-arrive) different test cases. Our tests don't need the same protections as our production code, and the increased clarity of highlighting which bits are the same between all the test cases is a win overall.

Now that the test compiles, you can run it from your favorite IDE or the command line. For the command-line test, our typical options of Gradle and Maven both provide easy ways to run tests (`gradle test` or `mvn test`).

NOTE IDEs also have their own individual ways of running JUnit tests, but generally speaking, they all allow you to right-click on the test class for a Run Test option. Once you do that, the IDE will display a window or section that informs you that your test has failed, because the expected value of 30 was not returned by the call to `estimateTotalRevenue(1)` ; instead, 0 was returned.

Now that you have a failing test, the next step is to make the test pass (go green).

Writing a passing test (green)

The point in this step is to make the test pass, but the implementation doesn't have to be perfect. By providing the `TicketRevenue` class with a better implementation of `estimateTotalRevenue()` (an implementation that doesn't just return 0), you'll make the test pass (go green).

Remember, at this stage, you're trying to make the test pass without necessarily writing perfect code. Your initial solution might look something like the following code:

```
import java.math.BigDecimal;

public class TicketRevenue {
  public BigDecimal estimateTotalRevenue(int numberOfTicketsSold) {
    BigDecimal totalRevenue = BigDecimal.ZERO;
    if (numberOfTicketsSold == 1) {
      totalRevenue = new BigDecimal("30");     ❶
    }

    return totalRevenue;
  }
}
```

❶ An implementation that passes the test

When you now run the test, it will pass, and in most IDEs, that will be indicated with a green bar or tick. Even our command lines give us a friendly green message to let us know all is well with the code.

The next question is, should you then say "I'm done!" and move on to the next bit of work? The resounding answer here should be "No!" Like us, you'll be itching to tidy up the previous code listing, so let's get into that right now.

Refactoring the test

The point of this step is to look at the quick implementation you wrote to pass the test and make sure that you're following accepted practice. Clearly the code isn't as clean and tidy as it could be. You can certainly refactor it and improve life for yourself and others in the future.

Remember, now that you have a passing test, you can *refactor without fear*. There's no chance of losing sight of the business logic that you've implemented so far.

TIP Another benefit that you've given yourself and the broader team by writing the initial passing test is a faster overall development process. The rest of the team can immediately take this first version of the code and begin to test it alongside the larger codebase (for integration tests and beyond).

In this example, you don't want to be using magic numbers—you want to make sure that the ticket price of 30 is a named concept in the code—so we write the following code:

```
import java.math.BigDecimal;

public class TicketRevenue {

  private final static int TICKET_PRICE = 30;                    (1)

  public BigDecimal estimateTotalRevenue(int numberOfTicketsSold) {
    BigDecimal totalRevenue = BigDecimal.ZERO;

    if (numberOfTicketsSold == 1) {
      totalRevenue = new BigDecimal(TICKET_PRICE *                (2)
                                    numberOfTicketsSold);
    }

    return totalRevenue;
  }
}
```

(1) No magic number

❷ Refactored calculation

The refactoring has improved the code, but clearly it doesn't cover all potential use cases (e.g., negative, `0`, `2-100`, and `> 100` ticket sales).

Instead of trying to guess what the implementation should look like for the other use cases, you should have further tests drive the design and the implementation. The next section follows test-driven design by taking you through more use cases in this ticket revenue example.

A TDD EXAMPLE WITH MULTIPLE USE CASES

One particular style of TDD would continue to add one test at a time for the negative, `0`, `2-100`, and `> 100` ticket sale test cases we presented before. But it's entirely valid to write a set of test cases up front, especially if they're related to the original test.

Note that it's still very important to follow the red-green-refactor lifecycle here. After adding all of these use cases, you might end up with a test class with failing tests (red) as follows:

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.math.BigDecimal;

import static org.junit.jupiter.api.Assertions.*;

public class TicketRevenueTest {

  private TicketRevenue venueRevenue;
  private BigDecimal expectedRevenue;

  @BeforeEach
  public void setUp() {
    venueRevenue = new TicketRevenue();
  }

  @Test
  public void failIfLessThanZeroTicketsAreSold() {                    ❶
    assertThrows(IllegalArgumentException.class,
            () -> venueRevenue.estimateTotalRevenue(-1));
  }
```

```
  @Test
  public void zeroSalesEqualsZeroRevenue() {                    ❷
    assertEquals(BigDecimal.ZERO, venueRevenue.estimateTotalRevenue(0));
  }

  @Test
  public void oneTicketSoldIsThirtyInRevenue() {                ❸
    expectedRevenue = new BigDecimal("30");
    assertEquals(expectedRevenue, venueRevenue.estimateTotalRevenue(1));
  }

  @Test
  public void tenTicketsSoldIsThreeHundredInRevenue() {         ❹
    expectedRevenue = new BigDecimal("300");
    assertEquals(expectedRevenue, venueRevenue.estimateTotalRevenue(10));
  }

  @Test
  public void failIfMoreThanOneHundredTicketsAreSold() {        ❺
    assertThrows(IllegalArgumentException.class,
            () -> venueRevenue.estimateTotalRevenue(101));
  }
}
```

❶ Negative sold case

❷ 0 sold case

❸ 1 sold case

❹ N sold case

❺ > 100 sold case

The initial basic implementation to pass all of those tests (green) would then look something like the following:

```
import java.math.BigDecimal;

public class TicketRevenue {
  public BigDecimal estimateTotalRevenue(int numberOfTicketsSold)
    throws IllegalArgumentException {

    if (numberOfTicketsSold < 0) {
```

```
      throw new IllegalArgumentException(                    ❶
              "Must be > -1");
    }

    if (numberOfTicketsSold == 0) {
      return BigDecimal.ZERO;
    }

    if (numberOfTicketsSold == 1) {
      return new BigDecimal("30");
    }

    if (numberOfTicketsSold == 101) {
      throw new IllegalArgumentException(                    ❷
              "Must be < 101");
    }

    return new BigDecimal(30 * numberOfTicketsSold);         ❸
  }
}
```

❶ Exceptional cases

❷ Exceptional cases

❸ N sold case

With the implementation just completed, you now have passing tests.

Again, by following the TDD lifecycle, you'll now refactor that implementation. For example, you could combine the illegal `numberOfTicketsSold` cases (< 0 or > 100) into one `if` statement and use a formula (`TICKET_PRICE * numberOfTicketsSold`) to return the revenue for all other legal values of `numberOfTicketsSold` . The following code should be similar to what you would come up with:

```
import java.math.BigDecimal;

public class TicketRevenue {

  private final static int TICKET_PRICE = 30;

  public BigDecimal estimateTotalRevenue(int numberOfTicketsSold)
    throws IllegalArgumentException {
```

```
    if (numberOfTicketsSold < 0 || numberOfTicketsSold > 100) {
      throw new IllegalArgumentException(                    ❶
                "# Tix sold must == 1..100");
    }

    return new BigDecimal(TICKET_PRICE *                     ❷
                  numberOfTicketsSold);
  }
}
```

❶ Exceptional case

❷ All other cases

The `TicketRevenue` class is now far more compact and yet still passes all of the tests! You've completed the full red-green-refactor cycle and can confidently move on to your next piece of business logic. Alternatively, you can start the cycle again, should you (or the accountant) spot any edge cases you've missed, such as having a variable ticket price.

13.4 Test doubles

As you continue to write code in a TDD style, you'll quickly run into the situation where your code references some (often third-party) dependency or subsystem. In this situation, you'll typically want to ensure that the code under test is isolated from that dependency to ensure that you're writing test code only against what you're actually building. You'll also want the tests to run as quickly as possible, especially if you're aiming to write unit rather than integration tests. Invoking a third-party dependency or subsystem, such as a database, can take a lot of time, which means you lose the fast feedback benefit of TDD. *Test doubles* are the solution to this problem.

In this section, you'll learn how a test double can help you to effectively isolate dependencies and subsystems. You'll work through examples that use the four types of test double (dummy, stub, fake, and mock). We'll also look at some of the perils and difficulties test doubles bring alongside their benefits.

We like Gerard Meszaros's simple explanation of a test double in his *xUnit Test Patterns* book (Addison-Wesley Professional, 2007), so we'll gladly quote him here: "A *Test Double* (think Stunt Double) is the generic term for any kind of pretend object used in place of a real object for testing purposes."

Meszaros defines four kinds of test doubles, which are outlined in table 13.1.

Table 13.1 The four types of test doubles

Type	Description
Dummy	An object that is passed around but never used; typically used to fulfill the parameter list of a method
Stub	An object that always returns the same canned response; may also hold some dummy state
Fake	An actual working implementation (not of production quality or configuration) that can replace the real implementation
Mock	An object that represents a series of expectations and provides canned responses

The four types of test doubles are far easier to understand when you work through code examples that use them. Let's go do that now, starting with the dummy object.

3.4.1 Dummy object

A *dummy* object is the easiest of the four test double types to use. Remember, it's designed to help fill parameter lists or fulfill some mandatory field requirements where you know the object will never get used. In many cases, you can even pass in an empty object (or even `null`, although this is not guaranteed to be safe).

Let's go back to the theater tickets scenario. It's all very good having an estimate of the revenue coming in from your single kiosk, but the owners of the theater have started to think a bit bigger. Better modeling of the tickets sold and the revenue expected is needed, and you hear murmurings of more requirements and complexity coming down the pipe.

You've been asked to keep track of the tickets sold, and to allow for a 10% discounted price on some tickets. It looks like you're going to need a `Ticket` class that provides a discounted price method. You start the familiar TDD cycle with a failing test, focusing on a new `getDiscountPrice()` method. You also know that there will need to be a couple of constructors: one for a regular-priced ticket, and one where the face value of the ticket may vary. The `Ticket` object will ultimately expect the following two arguments:

- *The client name*—A `String` that won't be referenced at all for this test
- *The normal price*—A `BigDecimal` that will get used for this test

You're pretty sure that the client name won't be referenced in the `getDiscountPrice()` method. This means you can pass the constructor a dummy object (in this case, the arbitrary string `"Riley"`), as shown in the following code:

```
import org.junit.jupiter.api.Test;

import java.math.BigDecimal;

import static org.junit.jupiter.api.Assertions.*;

public class TicketTest {

  private static String dummyName = "Riley";                    ❶

  @Test
  public void tenPercentDiscount() {
    Ticket ticket = new Ticket(dummyName,                       ❷
                               new BigDecimal("10"));

    assertEquals(new BigDecimal("9.0"), ticket.getDiscountPrice());
  }
}
```

❶ Creates a dummy object

❷ Passes in a dummy object

As you can see, the concept of a dummy object is trivial.

To make the concept extremely clear, the code in the following snippet has a partial implementation of the `Ticket` class:

```
import java.math.BigDecimal;

public class Ticket {

  public static final int BASIC_TICKET_PRICE = 30;              ❶
  private static final BigDecimal DISCOUNT_RATE =               ❷
                                  new BigDecimal("0.9");
```

```
  private final BigDecimal price;
  private final String clientName;

  public Ticket(String clientName) {
    this.clientName = clientName;
    price = new BigDecimal(BASIC_TICKET_PRICE);
  }

  public Ticket(String clientName, BigDecimal price) {
    this.clientName = clientName;
    this.price = price;
  }

  public BigDecimal getPrice() {
    return price;
  }

  public BigDecimal getDiscountPrice() {
    return price.multiply(DISCOUNT_RATE);
  }
}
```

❶ The default price

❷ The default discount

Some developers become confused by dummy objects—they look for complexity that doesn't exist. Dummy objects are very straightforward: they're any old object used to avoid `NullPointerException` and to get the code to run.

Let's move on to the next type of test double. The next step up (in terms of complexity) is the stub object.

3.4.2 Stub object

You typically use a *stub* object when you want to replace a real implementation with an object that will return the same response every time. Let's return to our theater ticket pricing example to see this in action.

You've come back from a well-deserved holiday after implementing the `Ticket` class, and the first thing in your inbox is a bug report stating that your `tenPercentDiscount()` test is now failing intermittently. When you look into the codebase, you see that the `Ticket` class is now using a concrete `HttpPrice` class that implements a newly intro-

duced `Price` interface. As the name suggests, `HttpPrice` contacts an external website and can return different values—or fail—at any point.

This is making the test fail, but further has polluted the purpose of our test. Remember, all you wanted was to unit-test calculating the 10% discount!

NOTE Calling a third-party pricing site is certainly not part of this test's responsibility. Separate integration tests should cover the `HttpPrice` class and its third-party `HttpPricingService`.

To get our test back to a consistent, stable point, we will replace the `HttpPrice` class with a stub instead. First, let's take a look at the current state of the code, as shown in the three following code snippets:

```
import org.junit.jupiter.api.Test;

import java.math.BigDecimal;

import static org.junit.jupiter.api.Assertions.*;

public class TicketTest {

  private static String dummyName = "Riley";

  @Test
  public void tenPercentDiscount() {
    Price price = new HttpPrice();                    (1)
    Ticket ticket = new Ticket(dummyName, price);     (2)
    assertEquals(new BigDecimal("9.0"),               (3)
                 ticket.getDiscountPrice());
  }
}
```

(1) HttpPrice implements Price.

(2) Creates Ticket

(3) The test can fail.

The next snippet shows the new implementation of `Ticket`:

```
import java.math.BigDecimal;

public class Ticket {
  private final String clientName;
  private final Price priceSource;
  private final BigDecimal discountRate;

  private BigDecimal faceValue = null;

  public Ticket(String clientName,
                Price price,
                BigDecimal discountRate) {          ❶
    this.clientName = clientName;
    this.priceSource = price;
    this.discountRate = discountRate;
  }

  public BigDecimal getPrice() {
    if (faceValue == null) {
      faceValue = priceSource.getInitialPrice();    ❷
    }

    return faceValue;
  }

  public BigDecimal getDiscountPrice() {
    return faceValue.multiply(discountRate);        ❸
  }
}
```

❶ The altered constructor

❷ The new getInitialPrice call

❸ The unchanged calculation

Providing a full implementation of the `HttpPrice` class would take us too far afield, so let's just suppose that it calls out to another class, `HttpPricingService`, as shown here:

```
import java.math.BigDecimal;

public interface Price {
  BigDecimal getInitialPrice();
}
```

```
public class HttpPrice implements Price {
  @Override
  public BigDecimal getInitialPrice() {
    return HttpPricingService.getInitialPrice();    ❶
  }
}
```

❶ Returns random results

Now that we've surveyed the damage, let's think about what we intended to test. Our goal was to show the multiplication in the `Ticket` class's `getDiscountPrice()` method works as expected. No external websites are required for proving that.

The `Price` interfaces gives us the seam we need to replace our touchy `HttpPrice` instance with a consistent `StubPrice` implementation, as follows:

```
import org.junit.jupiter.api.Test;

import java.math.BigDecimal;

import static org.junit.jupiter.api.Assertions.*;

public class TicketTest {
  @Test
  public void tenPercentDiscount() {
    Price price = new StubPrice();              ❶
    Ticket ticket = new Ticket(price);          ❷
    assertEquals(new BigDecimal("9.0"),         ❸
                 ticket.getDiscountPrice());
  }
}
```

❶ The StubPrice stub

❷ Creates a Ticket

❸ Checks the price

The `StubPrice` class is a simple little class that consistently returns the initial price of 10, as shown here:

```
import java.math.BigDecimal;

public class StubPrice implements Price {
  @Override
  public BigDecimal getInitialPrice() {
    return new BigDecimal("10");          (1)
  }
}
```

(1) Returns a consistent price

Phew! Now the test passes again, and, equally important, you can look at refactoring the rest of the implementation details without fear.

Stubs are a useful type of test double, but sometimes it's desirable to have the stub perform some real work that's closer to the production system. For that, you use a fake object as your test double.

3.4.3 Fake object

A *fake* object can be seen as an enhanced stub that almost does the same work as your production code, but that takes a few shortcuts to fulfill your testing requirements. Fakes are especially useful when you'd like your code to run against something that's very close to the real third-party subsystem or dependency that you'll use in the live implementation.

For our ticketing application, let's imagine that our database layer provides us with a simple interface for working with tickets, shown next:

```
package com.wellgrounded;

public interface TicketDatabase {
    Ticket findById(int id);
    Ticket findByName(String name);
    int count();

    void insert(Ticket ticket);
    void delete(int id);
}
```

Our class for managing an individual show needs to work with this database interface and manage features such as checking we haven't oversold like this:

```
package com.wellgrounded;

import java.math.BigDecimal;

public class Show {
   private TicketDatabase db;
   private int capacity;

   public Show(TicketDatabase db, int capacity) {
      this.db = db;
      this.capacity = capacity;
   }

   public void addTicket(String name, BigDecimal amount) {
      if (db.count() < capacity) {
         var ticket = new Ticket(name, amount);
         db.insert(ticket);
      } else {
        throw new RuntimeException("Oversold");
      }
   }
}
```

We'd like to unit test `addTicket` without relying on a fully instance of our relational database. Such a test might look like this:

```
package com.wellgrounded;

import org.junit.jupiter.api.Test;
import java.math.BigDecimal;
import static org.junit.jupiter.api.Assertions.*;

public class ShowTest {
   @Test
   public void plentyOfSpace() {
      var db = new FakeTicketDatabase();      1
      var show = new Show(db, 5);

      var name = "New One";
      show.addTicket(name, BigDecimal.ONE);
```

```
        var mine = db.findByName(name);
        assertEquals(name, mine.getName());
        assertEquals(BigDecimal.ONE, mine.getAmount());
    }
}
```

❶ FakeTicketDatabase doesn't exist, but in the spirit of TDD, we'll write the code we want to pass.

Although we could accomplish this via stubbing, it would have major drawbacks. We'd have to stub out the methods `count` and `insert` on the database, which aren't even visible in the test, cluttering it up with lower-level details and distracting from its actual purpose. But the difficulties go deeper still—each test must ensure the relationship between `count` and the number of calls to `insert` are aligned. Even worse, our final call to `findByName`, which is meant to ensure our data was saved, would also need to be stubbed. But that very stubbing means the assertion is useless—it would pass whether or not our implementation code was correct! Stubs fall short of letting us accurately validate this set of tightly related actions.

A fake object provides an alternative by having a real but simplified, implementation. The provided interface, shown next, is easy to serve with a wrapper around a simple `HashMap`:

```
package com.wellgrounded;

import java.util.HashMap;

class FakeTicketDatabase implements TicketDatabase {
    private HashMap<Integer, Ticket> tickets =          ❶
                            new HashMap<>();
    private Integer nextId = 1;                         ❷

    @Override
    public Ticket findByName(String name) {
        var found = tickets.values()
                .stream()
                .filter(ticket -> ticket.getName().equals(name))
                .findFirst();
        return found.orElse(null);
    }

    @Override
    public int count() {
```

```
        return tickets.size();
    }

    @Override
    public void insert(Ticket ticket) {
        tickets.put(nextId, ticket);
        nextId++;
    }

    // Remaining methods available in resources
}
```

❶ Our map takes the place of a database for the lifetime of our unit test.

❷ We have to replicate features such as databases autoincrementing IDs.

Fake objects, especially when shared throughout a project with strong interfaces, can be a nice solution for supporting unit tests. They aren't appropriate everywhere—if our database interface allowed us to pass SQL clauses for additional filtering, this would quickly get beyond our fake's ability to handle—but they're a useful tool to have. Just keep an eye out that the implementation doesn't get too large or complex, because every line of code we write is a possible source of bugs.

3.4.4 Mock object

Mock objects are related to the stub test doubles that you've already met, but stub objects are usually pretty dumb beasts. For example, stubs typically fake out methods to always give the same result. This doesn't provide any way to model state-dependent behavior.

As an example: you're trying to follow TDD, and you're writing a text analysis system. One of your unit tests instructs the text analysis classes to count the number of occurrences of the phrase "Java11" for a particular blog post. But because the blog post is a third-party resource, a number of possible failure scenarios exist that have very little to do with the counting algorithm you're writing. In other words, the code under test isn't isolated, and calling the third-party resource could be time-consuming. Here are some common failure scenarios:

- Your code might not be able to go out to the internet to query the blog post, due to firewall restrictions in your organization.
- The blog post may have been moved with no redirect.

- The blog post might be edited to increase or decrease the number of times "Java11" appears.

Using stubs, this test would be almost impossible to write, and it would be incredibly verbose for each test case. Enter the *mock object*. This is a special kind of test double, which you can think of as a programmable stub. Using the mock object is very simple: when you're preparing the mock for use, you tell it the sequence of calls to expect and how it should respond to each one.

Let's see this in action by looking at a simple example for the theater tickets use case. We'll be using the popular mocking library, Mockito (**https://site.mockito.org**). The following snippet shows how to use it:

```
import org.junit.jupiter.api.Test;
import java.math.BigDecimal;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

public class TicketTest {
  @Test
  public void tenPercentDiscount() {
    Price price = mock(Price.class);      ❶

    when(price.getInitialPrice())
      .thenReturn(new BigDecimal("10"));  ❷

    Ticket ticket = new Ticket(price, new BigDecimal("0.9"));
    assertEquals(new BigDecimal("9.0"), ticket.getDiscountPrice());

    verify(price).getInitialPrice();
  }
}
```

❶ Creates a mock

❷ Programs the mock for testing

To create a mock object, you call the static `mock()` method with the class object of the type you want to mock up. Then you "record" the behavior that you want your mock to display by calling the `when()` method to indicate which method to record, and `thenReturn()` to specify the expected result. Last, you verify that you've called the expected

methods on the mocked object. This ensures that you didn't get to the correct result via an incorrect path.

This verification captures the big difference between a stub and a mock. With a stub, your primary focus is returning canned values. With a mock, the intent is *verifying behavior*, such as the exact calls actually made. In practice, Mockito's richly featured `mock()` method can be used easily to create stubs if we ignore verification, but it's important that you as the programmer be aware of what you're intending to test.

You can use the mock just like a regular object and pass it to your call to the `Ticket` constructor without any further ceremony. This makes mock objects a very powerful tool for TDD. Some practitioners don't really use the other types of test doubles, preferring to do almost everything with mocks. But as with many powerful tools, mocking comes with sharp edges to be aware of.

3.4.5 Problems with mocking

One of the largest difficulties with test doubles is precisely that they are fake, so their behavior can diverge from the actual production system. Unfortunately, this can happen while still leaving you with the warm, comforting sense that your tests have you fully covered—until reality bites.

These behavior differences can come in a number of flavors. Common ones include the following:

- Differences in returned payloads, particularly with complex nested objects
- Serialization/deserialization differences from testing data
- Ordering of items in collections
- Responses to error conditions—either failing to throw, or throwing different exception types

Although there's no blanket solution, these problems can often be spotted when we step up a level to integration tests. Again, if we keep firmly in mind *what* we're testing in each set of tests, we can focus our unit tests on localized logic and use integration testing elsewhere to cover interactions with our dependencies.

Solid design of our interfaces also helps our test doubles. Rather than a service class returning a raw contents string from an HTTP call, returning a specific object gives our test doubles less room to vary. Having precise subclasses for the exceptions your class raises, wrapping more primitive exception types, not only makes your code more expressive but also easier to mock accurately.

Mocking, if used everywhere, can also lead to our tests too closely mimicking our production code. When this happens, each line in your actual code has a mirroring line in test configuration, restating the precise expected call. Such tests are extremely brittle, often leading to huge swathes of seemingly unrelated changes when altering code.

This brittleness when mocking extends down to the level of individual arguments. Although the frameworks make it easy to be extremely precise about the values being passed, consider whether your test truly needs to verify that argument. As we saw with dummy objects, for some test cases, a given value might not matter. Mocking frameworks provide mechanisms for effectively allowing statements like "any integer," and if the value doesn't matter, such statements both clarify what your test cares about and allows room for the production code to evolve more easily. Give your tests room to breathe!

Tests can also surface issues when they require large amounts of intricate setup before running. Mocking, particularly when paired with dependency injection, can make it easy to pile up dependencies in a class without taking much notice. If your test setup is longer than the code needed to execute and verify your results, that's a hint your classes may be overly complex and ripe for refactoring. Test setup is also a great way to watch for violations of the Law of Demeter, which suggests objects should have knowledge only of their direct neighbors. If your test setup needs to muck about with objects many levels removed from itself, your objects might be reaching too far outside themselves.

Test doubles are a valuable tool for the well-grounded developer. We've demonstrated a little bit of JUnit in our discussion so far, but we have not explored it in depth yet. Let's look a little closer at it and take the opportunity to see what's new with JUnit 5, the most recent major version.

13.5 From JUnit 4 to 5

JUnit is a JVM-based implementation of the xUnit style of test framework, developed initially by Kent Beck and Erich Gamma. This flavor of unit testing has proven versatile and easy to use, placing JUnit among the most commonly used libraries in the JVM ecosystem.

This long history and broad usage brings many constraints as well. JUnit 4, initially released in 2006, couldn't use features like lambda expressions without breaking compatibility. In 2017, JUnit 5 was released, using the opportunity afforded by the major version change to introduce significant changes.

NOTE The next chapter will focus plenty of time on other tools and techniques, but the well-grounded Java developer will almost certainly encounter JUnit code that's worth moving to the most modern versions.

One of the biggest changes with JUnit 5 is its packaging. Where prior versions were monolithic, containing both the API for authoring tests and the support for running and reporting on those tests, JUnit 5 breaks things down into more focused packages. JUnit 5 also steps away from the external dependency on Hamcrest that came with JUnit 4.

JUnit 5 lives in an entirely new package— `org.junit.jupiter` —which means both version can coexist during migration. We'll look more at the mechanics of that in a moment.

JUnit 5's primary two dependencies follow:

- `org.junit.jupiter.junit-jupiter-api` —This is referenced from your testing code to provide all the necessary annotations and helpers to author tests.
- `org.junit.jupiter.junit-jupiter-engine` —This is the default engine for running JUnit 5 tests. It is only needed as a runtime, not compile-time, dependency and can be augmented or swapped out for other test runners.

In Gradle, that would look like this, along with a hint to tell Gradle to use the new JUnit bits when running tests:

```
dependencies {
 testImplementation("org.junit.jupiter:junit-jupiter-api:5.7.1")
 testRuntimeOnly("org.junit.jupiter:junit-jupiter-engine:5.7.1")
}

tasks.named<Test>("test") {
 useJUnitPlatform()
}
```

The equivalent for Maven would be as follows. The Maven `surefire` and `failsafe` plugins know how to work automatically with JUnit 5, as long as you have a new-enough version (2.22 or greater is recommended):

```
<project>
 <dependencies>
  <dependency>
   <groupId>org.junit.jupiter</groupId>
   <artifactId>junit-jupiter-api</artifactId>
```

```
      <version>5.7.1</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-engine</artifactId>
      <version>5.7.1</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

If you add these to a JUnit 4 project and simply run the tests, you'll find an odd outcome—the suite will likely pass—but if you look more closely at the reports, *no tests actually ran!* This is because the actual annotation to mark test cases changed with JUnit 5.

NOTE JUnit 5 brings its own `@Test` annotation in the package `org.junit.jupiter.api`. It won't recognize existing tests marked with the older `org.junit` version of `@Test` by default!

There are two paths to take from this point. The first is to change over each place where you're importing the old annotations to use the new version. Class by class, your test suite will start running under JUnit 5. These conversions may require other work we'll discuss shortly.

An alternative is to pull in an additional runtime dependency on the `junit-vintage-engine`. This package uses JUnit 5's richer ability to plug in different runners and support classes to allow for backward compatibility with JUnit 4 (and even 3) tests.

In Gradle:

```
dependencies {
  testImplementation("org.junit.jupiter:junit-jupiter-api:5.7.1")
  testRuntimeOnly("org.junit.jupiter:junit-jupiter-engine:5.7.1")

  testRuntimeOnly("org.junit.vintage:junit-vintage-engine:5.7.1")
}
```

In Maven:

```
<project>
 <dependencies>
  <dependency>                          (1)
   <groupId>junit</groupId>
   <artifactId>junit</artifactId>
   <version>4.13</version>
   <scope>test</scope>
  </dependency>
  <dependency>
   <groupId>org.junit.vintage</groupId>
   <artifactId>junit-vintage-engine</artifactId>
   <version>5.7.1</version>
   <scope>test</scope>
  </dependency>
  <dependency>                          (2)
   <groupId>org.junit.jupiter</groupId>
   <artifactId>junit-jupiter-api</artifactId>
   <version>5.7.1</version>
   <scope>test</scope>
  </dependency>
  <dependency>
   <groupId>org.junit.jupiter</groupId>
   <artifactId>junit-jupiter-engine</artifactId>
   <version>5.7.1</version>
   <scope>test</scope>
  </dependency>
 </dependencies>
</project>
```

(1) Support for running JUnit 4 tests alongside JUnit 5

(2) Our main JUnit 5 dependencies, api and engine

This support can allow for an easier transition as you can enable JUnit 5, then convert test by test over time, rather than requiring everything to move at once. It's worth noting, though, that the vintage support does have some limitations, which are well documented in the JUnit users guide at **http://mng.bz/5Q61**.

Along with the new packaging, a variety of classes have been renamed to make them clearer and more accurate about their usage, shown here:

- `@Before` changed to `@BeforeEach`.
- `@After` changed to `@AfterEach`.

- `@BeforeClass` changed to `@BeforeAll`.
- `@AfterClass` changed to `@AfterAll`.
- `@Category` changed to `@Tag`.
- `@Ignored` changed to `@Disabled` (or may be handled with `ExecutionCondition` in the new extension model).
- `@RunWith`, `@Rule` and `@ClassRule` are replaced by the new extension model.

As the final couple of points allude to, a big feature of JUnit 5 is a new extension model that covers a variety of separate features in earlier versions of JUnit. These features let you share behavior between classes—steps like test setup, teardown, and expectation setting—but didn't all fit together cohesively.

As an example, here's a basic JUnit 4 test that needs to start up a server before tests can run. It uses the `ExternalResource` class, along with a `@Rule` annotation to ask that it be called at the right points in the lifecycle:

```
package com.wellgrounded;

import org.junit.Rule;
import org.junit.Test;
import org.junit.rules.ExpectedException;
import org.junit.rules.ExternalResource;

import static org.junit.Assert.*;

public class PasswordCheckerTest {
  private PasswordChecker checker = new PasswordChecker();

  @Rule                                    ❶
  public ExternalResource passwordServer =
                  new ExternalResource() {  ❷
    @Override
    protected void before() throws Throwable {
      super.before();
      checker.reset();
      checker.start();
    }

    @Override
    protected void after() {
      super.after();
      checker.stop();
    }
  };
```

```
    @Test
    public void ok() {
      assertTrue(checker.isOk("abcd1234!"));
    }
}
```

❶ @Rule requests that this be applied before/after each test.

❷ ExternalResource is provided by JUnit specifically for such custom setup/teardown scenarios.

Our overrides on `ExternalResource` could easily be pulled into another location and shared between our tests.

JUnit 5 instead breaks down the testing lifecycle into smaller interfaces that you implement. These extensions can then be applied at the class or test-method level, as shown next:

```
package com.wellgrounded;

import org.junit.jupiter.api.extension.AfterEachCallback;
import org.junit.jupiter.api.extension.BeforeEachCallback;
import org.junit.jupiter.api.extension.ExtensionContext;

public class PasswordCheckerExtension
    implements AfterEachCallback, BeforeEachCallback {                    ❶

    private PasswordChecker checker;

    PasswordCheckerExtension(PasswordChecker checker) {                    ❷
      this.checker = checker;
    }

    @Override
    public void beforeEach(ExtensionContext context) {                     ❸
      checker.reset();
      checker.start();
    }

    @Override
    public void afterEach(ExtensionContext context) {                      ❸
      checker.stop();
```

```
    }
}
```

❶ We implement AfterEachCallback and BeforeEachCallback to get called as we did before for each test method. AfterAllCallback and BeforeAllCallback exist as well to replace @ClassRule functionality.

❷ Because our extension worked with a field on the test class, we need to have that passed on construction.

❸ Callbacks proceed as before to do the setup/teardown work.

With this class in hand, we can apply it in our test as follows:

```
package com.wellgrounded;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.RegisterExtension;

import static org.junit.Assert.*;

public class PasswordCheckerTest {
    private static PasswordChecker checker = new PasswordChecker();

    @RegisterExtension                                    ❶
    static PasswordCheckerExtension ext =
            new PasswordCheckerExtension(checker);                ❷

    @Test
    public void ok() {
        assertTrue(checker.isOk("abcd1234!"));
    }
}
```

❶ @RegisterExtension allows us to instantiate our extension for the test.

❷ The field on the test class must be public to work with @RegisterExtension.

If an extension doesn't require parameters on construction, it can also be applied to the class or method definition using `@ExtendWith`, as shown here:

```
@ExtendWith(CustomConfigurationExtension.class)
public class PasswordCheckerTest {
   // ....
}
```

JUnit 5's move to require more recent JDK versions also tidied up some corners where rules used to be a standard approach. Checking whether a test method raised an exception could take the following two forms in JUnit 4 and earlier:

```
package com.wellgrounded;

import org.junit.Rule;
import org.junit.Test;
import org.junit.rules.ExpectedException;

import static org.junit.Assert.*;

public class PasswordCheckerTest {
   private PasswordChecker checker = new PasswordChecker();

   @Rule
   public ExpectedException ex = ExpectedException.none();

   @Test
   public void nullThrows() {
      ex.expect(IllegalArgumentException.class);     ❶
      checker.isOk(null);
   }

   @Test(expected = IllegalArgumentException.class)   ❷
   public void alsoThrows() {
      checker.isOk(null);
   }
}
```

❶ A rule-based exception checking on a test method

❷ The @Test annotation configuration of the excepted exception

Lambda expressions open up a new way of expressing this that looks more like our typical assertions, like this:

```
package com.wellgrounded;

import org.junit.jupiter.api.Test;

import static org.junit.Assert.*;

public class PasswordCheckerTest {
   private static PasswordChecker checker = new PasswordChecker();

   @Test
   public void nullThrows() {
      assertThrows(IllegalArgumentException.class, () -> {
         checker.isOk(null);
      });
   }
}
```

`assertThrows` is preferable to the old `expected` argument on our test annotations or the `ExpectedException` rule for a couple of reasons. First, the assertion is more direct, located right where the code we're actually testing is found instead of earlier in the method. Also, `assertThrows` returns the exception without us needing to scaffold a `try` / `catch` in place, so we can more easily do typical assertions about what happened without special plumbing. Although JUnit 4's legacy across the ecosystem means it'll still be around—and maintained—for a long while, if you're using JUnit, it's worth looking at what the new version offers.

Our test libraries are only part of the picture, though, especially once we're writing integration tests. The next chapter will delve into some helpful tools in approaching the long-standing hassles of testing with external dependencies.

Summary

- We've discussed the motivations and types of testing, and how it's critical to know the answers to what we're testing for to decide on an approach.
- We took a walk through test-driven development, seeing how it allows us to evolve a design with confidence in a step-by-step fashion.
- We've examined the wide variety of test doubles, what they're useful for, and, more important, where they'll cause trouble if misused.
- After many years, a new version of JUnit is out that addresses long-standing design problems, often constraints from long-deprecated JDKs. We looked briefly at moving to this latest version for the basics of our testing.

Previous chapter

< 12 Running Java in containers

Next chapter

14 Testing beyond JUnit >